

Securing Base Protocols

Ari Bender-Long, Xavier Cadot, Andrew Le, Alejandro Mestanza

Overview

- Securing base protocols
 - Implement a hardware or software solution that mitigates or eliminates a vulnerability in a base protocol such as TCP, UDP, DNS, etc.
- We chose to look at the Neighbor Discovery Protocol (NDP)
 - IPv6 protocol
 - Developed in 2007
 - RFC 4861 - <https://tools.ietf.org/html/rfc4861>
 - Chosen because IPv6 is relatively new



Background

- NDP is a link layer protocol for IPv6 and is used for gathering routing information on the local network
 - IPv6 version of ARP (Address Resolution Protocol)
- NDP shares many of the same vulnerabilities as ARP
 - Including Spoofing, DoS, Cache poisoning
- NDP defines 5 ICMPv6 message types
 - Router Solicitation (RS)
 - Router Advertisement (RA)
 - Neighbor Solicitation (NS)
 - Neighbor Advertisement (NA)
 - Redirect



NDP Spoofing

- Like ARP, there is not any authentication with address resolution
- Malicious actor crafts RA or NA messages and sends them to a victim or broadcasts it to the network
- Detectable since mappings are not supposed to change often and advertisements are generally not spammed
 - Some spoofing attacks spam crafted messages

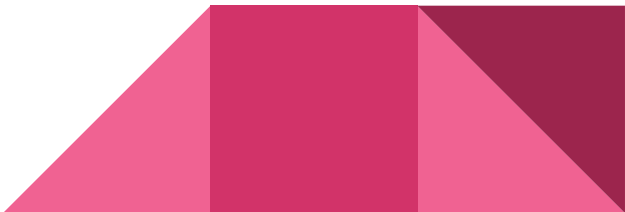


Design

- We are developing a tool that monitors incoming NDP packets
 - Checks the source against a list of known sources to ensure that a separate entity is not spoofing another device
 - On initialization, either query the system for the list of neighbors using “ip -6 neighbor” or take the initial advertisements and store them
 - We can try to detect attempts to change the mapping or if there are too many attempts (some actors try to spam spoofed packets to poison the cache)
 - Upon detection, we generate a detailed error message that we can use to pipe into a SIEM or logging solution



Progress

- Using Python 3 for codebase
 - Wrote the base code for monitoring and processing of packets
 - Gets, prints, and counts the incoming and outgoing ICMPv6 packets
 - Researched and prototyped code for running the program as a background service in both Linux and Windows
 - Linux: Added monitoring functionality, working on integrating as systemd service
 - Windows: Added logging functionality, wrote packet sniffer as a Windows Service
- 

Future Work

- Finish the rest of the base functionality
 - Look into netfilter and BPF (Berkeley Packet Filter) for possible aggressive blocking
 - Add in log generator - Linux
 - Finish packet sniffing on Windows
- Look at SeND specification to develop plan for implementation
- Create deploy scripts if necessary
- Create test attack emulation scripts



Future Work (cont.)

- Implement parts of the Secure Neighbor Discovery (SeND)
 - Need a certificate generator that crafts both valid and invalid packets with certificates
 - The client will need to verify the certificate using either a CA or local certificates
- Add the NDP spoofing detection to OSSEC
- Fix IPv6 issues that may arise in Windows environments
- Look for other vulnerable protocols and implement solutions for them
 - Combine the solution with the NDP tool





Questions?